LifeOS Overall System Check: UX and Performance

Nov 7, 2026 · @Deepthi

Overview

This check finds what makes LifeOS hard or slow to use and says how to fix it. It has three parts: the task, deadline and subtask experience, which you reported as hard to use; an overall UX check of every screen; and a performance and speed plan.

What it is based on. The LifeOS plans and your report that tasks with deadlines and subtasks work but feel bad. I have not used the live app, so each problem below is a likely cause to confirm with the test scenarios in the "How to run the check" section, not a measured result. Targets are proposals to tune, not facts about today's speed.

How to read it. Each section lists the problem, why it hurts and the fix. The first section after this one ranks what to fix first, and the last section turns the fixes into a plan by week.

Fix first

Ten fixes give most of the gain. They are ordered by impact against effort, with the cheap high-impact ones first.

#	Fix	Area	Impact	Effort
1	Add database indexes and paginate long lists (events, tasks, agent log)	Speed	High	Low
2	One-step task entry: title, deadline and subtasks in a single quick-add	Tasks UX	High	Medium
3	Group the 21 domains into 5 clusters with search and pinned favorites	Navigation	High	Low
4	Optimistic updates, so ticking a task or check-in responds instantly	Speed and UX	High	Low
5	Offer work blocks automatically when a task gets a deadline	Tasks UX	High	Medium
6	Route to the domain first, then send only that domain's tools to the model	Speed	High	Medium

#	Fix	Area	Impact	Effort
7	One notification budget across all domains	UX	High	Low
8	Load FullCalendar and domain modules only when opened	Speed	Medium	Low
9	Start onboarding with 4 or 5 domains, not 21	UX	Medium	Low
10	Overdue triage: one screen to reschedule, split or drop late tasks	Tasks UX	Medium	Medium

Tasks, deadlines and subtasks

The task feature works but is slow to use because creating and managing a deadline with subtasks takes too many steps, and the task list is cut off from the calendar. The likely problems and their fixes:

Likely problem	Why it hurts	Fix
Adding a task with a deadline and subtasks takes several screens	People stop capturing tasks	One quick-add: type "Report due Friday, subtasks: outline, draft, edit" in the bar or chat and get the parent with its subtasks, or add them inline on one sheet
A full date picker opens every time	Slow on a phone	Date chips (Today, Tomorrow, This week, Next week, Pick date) and typed dates like "Fri"
Subtasks are hidden or nested several levels deep	You cannot see progress	Two levels at most, subtasks as an inline checklist under the parent, and a progress ring (2 of 5) on the parent
A subtask can be due after its parent	Deadlines contradict each other	Rule: a subtask's due date can never be later than its parent's; it defaults to the parent's and the app warns on a conflict
A deadline does not create time in the calendar	The task stays a wish	When a task has a deadline and an estimate, offer suggested work blocks (the same backward planning used for exams)

Likely problem	Why it hurts	Fix
Ticking the parent leaves subtasks open	The list looks wrong	Ticking the parent asks to complete all subtasks; completing the last subtask offers to complete the parent
Overdue tasks pile up	The list turns into guilt	An overdue triage screen with Move, Split or Drop for each task and a suggested new date
Long lists	Hard to scan	Default view of the next 3 tasks plus those due soon, with filter chips; full grouping lives on each domain page
Editing takes many taps	Friction on every change	Swipe to complete or move to tomorrow, long-press to edit, bulk select, and an Undo toast
No time estimate	The planner cannot size blocks	Optional estimate chips (30 min, 1 h, 2 h, half a day); the agent proposes one if you skip it

New add-task flow

1. Tap the quick-add (or type in chat) and enter the title.
2. Tap a date chip; the estimate chips appear next.
3. Tap "Add subtasks" and type one per line; they default to the parent's deadline.
4. Save. The task shows with its progress ring, and a banner asks "Add work blocks?"
5. Accept creates suggested blocks through the calendar service; Undo removes them.

What changes in the data. The Task table gets `parentId`, `position` and a depth limit of 2; the rules `validate_subtask_due` and `rollup_progress` are plain tested functions. No new table is needed.

Done check. On a phone, adding a task with a deadline and 3 subtasks takes one sheet and no more than a few taps, and the parent's ring updates as subtasks are ticked.

Overall UX check

The biggest UX risk is scale: the app was designed for 12 domains and now has 21, with more reminders, more screens and more choices on every page. The check below covers each screen, then what cuts across them.

Screen	Check	Pass when
Today	Shows only what matters now: the brief, next 3 tasks, today's timeline	Nothing needs scrolling to find the next action
Calendar	Layers and color chips stay usable with 21 domains	Layers are grouped by the 5 clusters, with a one-tap "show only" and a saved view
Chat	Replies are short, show what changed, and Undo is visible	Every change card has Undo, and confirmations name the conflict in one sentence
Domains	Finding one of 21 domains is quick	Clusters, search and pinned favorites reach any domain in 2 taps
Domain page	Every domain has the same layout	Goals, tasks, notes and mini calendar sit in the same place on all 21 pages
Weekly review	One decision at a time	Accept, Edit and Skip are always visible, and the page is a few screens, not a wall
Settings	Sleep, limits, notification times and sensitive-domain switches are easy to find	Grouped by topic with plain-language labels
Onboarding	Does not ask for everything at once	Starts with 4 or 5 domains and lets you add more later

Across all screens

- **Notification budget.** 21 domains can each send reminders. Set one daily cap, merge reminders into a single digest at a time you choose, and let each domain be muted. Tier 1 anchors, such as doses, are the exception.
- **Confirmation fatigue.** Group several protected-time questions into one preview instead of asking one by one.
- **Instant feedback.** Every tap responds within a moment: optimistic updates, skeleton loaders and visible progress for the model's reply.
- **States.** Every screen has designed empty, loading and error states with a next step.
- **Consistency.** Same colors, spacing, icons and wording for the same action everywhere; one word per concept (task, not to-do or item).
- **Accessibility.** 44px touch targets, one-handed reach on a phone, readable contrast in light and dark mode, screen reader labels, and reduced motion respected.

- **Plain language.** Labels describe what happens ("Add work blocks"), not how the system works ("Run plan_backwards").

Performance and speed audit

Measure first, then fix. These are the numbers to track; the targets are proposals to confirm against real use. The first one comes from the original project proposal.

Metric	Proposed target	How to measure
Chat message to calendar updated	Under 5 seconds	Timestamps at each step stored in AgentLog: received, model start, tool run, write, reply
First word of a streamed reply	Under 1.5 seconds	Same timestamps, plus browser timing
Tick a task or check-in	Feels instant (under 100 ms)	Browser performance trace on a phone
Week view with a busy month	Under 1 second to usable	Lighthouse and web vitals on a mid-range phone
Open a domain page	Under 1 second	Web vitals; route timing on Vercel
Slowest database queries	95% under 100 ms	`pg_stat_statements` and `EXPLAIN ANALYZE`
Morning brief ready	Before wake time plus 30 minutes	Job duration logged per run

Where time is likely lost

- **Model round trips.** Each tool call in the loop is another model call, so a command that reads, checks and writes can take several.
- **Prompt size.** Sending all tools, all domain prompts and long history to the model on every message adds delay and cost, and gets worse with 21 domains.
- **Memory recall.** An embedding lookup on every message, even when no notes are needed.
- **Unindexed queries.** Events, tasks and the agent log grow every day; filters by user, date and domain need indexes.
- **Task queries.** Loading subtasks one parent at a time (the N+1 pattern) slows the task list.

- **Serverless cold starts and connections.** A new database connection on every request is slow on a hosted Postgres.
- **Heavy front end.** The calendar library and every domain module loaded on first open, and 21 layers rendered at once.
- **Background jobs.** Brief and review jobs that scan every user's data instead of only what changed.

Speed improvements

Area	Improvement	What it fixes
Database	Indexes on Event (user, start, domain), Task (user, due, parent) and AgentLog (user, created); paginate the agent log and long task lists	Slow lists and queries that grow with use
Database	Load a task with its subtasks in one query, not one per parent	Slow task list
Database	Use the host's pooled connection string for serverless	Connection setup on every request
AI	Route to the domain first with a small, cheap model, then send only that domain's tools and prompt to the model	Large prompts and slow replies with 21 domains
AI	Use a small model for parsing intent and a larger one only for planning and reviews, as the proposal already says	Cost and latency
AI	Cache the stable part of the prompt (rules, tool list) and keep only date, timezone and notes changing	Repeated prompt cost
AI	Run independent tool calls together, and skip memory recall for simple commands such as "undo"	Extra round trips
AI	Stream the reply and show "working" steps as they happen	Waiting with no feedback
Front end	Optimistic updates for ticks, check-ins and drag edits, with rollback on error	Delay on every tap

Area	Improvement	What it fixes
Front end	Load FullCalendar and each domain module only when opened; prefetch the next likely page	Heavy first load
Front end	Render only visible events and collapsed layer groups; virtualize long lists	Slow week and month views
Jobs	Precompute the brief and review inputs the night before and process only changed data	Slow or late background jobs
Caching	Cache read-only data such as domain config and the day's computed prayer times for a short time	Repeated work

Rule for every fix. Change one thing, measure the same metric before and after, and keep the change only if the number moves.

How to run the check

Run these seven scenarios on a real phone before any fix and again after it, so every change is compared with a baseline.

#	Scenario	Record	Pass when
1	Add a task with a deadline and 3 subtasks	Seconds and taps	One sheet, under 20 seconds (proposed)
2	Clear 5 overdue tasks	Taps per task	Each task handled in one tap plus a date choice
3	Open Karate, Finance and Medicine starting from Today	Taps each	No more than 2 taps each
4	Send 10 typical chat commands	Seconds to calendar update	Median under 5 seconds
5	Open the week view with a month of seeded events (hundreds)	Seconds to usable	Under 1 second
6	Turn on 10 domains for a week	Notifications per day	At most the daily cap you set

#	Scenario	Record	Pass when
7	Use the app for 7 days in a row	Friction log	Fewer than 3 high-severity items open

Friction log. For scenario 7, write one line each time something slows you down: date, screen, what happened, taps or seconds lost, and severity (low, medium, high). Fix high items first.

Tools

- Timestamps in AgentLog for chat latency.
- Lighthouse and web vitals for page speed on a mid-range phone.
- Vercel analytics for route timing and cold starts.
- `pg_stat_statements` and `EXPLAIN ANALYZE` for slow queries.
- Playwright scripts to repeat scenarios 1 to 5 and store the timings.
- Screen recording for tap counts in scenarios 1 to 3.

Fix plan

The fixes take 4 weeks, with measuring first so every later change has a baseline. Estimates assume one builder.

Week	Work	Done check
1. Measure and quick wins	Run the 7 scenarios for a baseline, add AgentLog timestamps, add the database indexes and pagination, group the domains into clusters with search and pinned favorites	Baseline numbers saved; any domain reachable in 2 taps; slowest queries measured again
2. Tasks UX	Quick-add with date and estimate chips, inline subtasks with a progress ring, subtask due-date rule, parent and subtask completion rules, swipe actions and Undo	Scenario 1 passes; ticking subtasks updates the parent instantly
3. Speed	Route-then-tools for the model, prompt caching, optimistic updates, lazy-loaded calendar and domain modules, one-query subtasks, pooled connections	Scenario 4 median under 5 seconds; week view under 1 second

Week	Work	Done check
4. Calm and polish	Notification budget and digest, grouped confirmations, overdue triage, onboarding with 4 or 5 domains, accessibility pass, then rerun all 7 scenarios	All scenarios pass or have a written reason; friction log has fewer than 3 high items

Keep it from coming back. Add the 7 scenarios to the release checklist, keep the timing logs on, and rerun the check each time a wave of new domains ships.